grep-cmp-challenges

Practical Linux grep command examples

First create the following demo_file that will be used in the examples below to demonstrate grep command.

$ cat demo_file

THIS LINE IS THE 1ST UPPER CASE LINE IN THIS FILE.

this line is the 1st lower case line in this file.

This Line Has All Its First Character Of The Word With Upper Case.

Two lines above this line is empty.

And this is the last line.

1. **Search for the given string in a single file:**
    a) $ grep "this" demo_file.
2. **Checking for the given string in multiple files:** For this example, let us copy the demo_file to demo_file1. The grep output will also include the file name in front of the line that matched the specific pattern as shown below. When the Linux shell sees the meta character, it does the expansion and gives all the files as input to grep.
    a) $ grep "this" demo_*
3. **Case insensitive search using grep -i:** This is also a basic usage of the grep. This searches for the given string/pattern case insensitively. So it matches all the words such as “the”, “THE” and “The” case insensitively as shown below.
    a) $ grep -i "the" demo_file
4. **Match regular expression in files:** This is a very powerful feature, if you can use use regular expression effectively. In the following example, it searches for all the pattern that starts with “lines” and ends with “empty” with anything in-between. i.e To search “lines[anything in-between]empty” in the demo_file.

a) $ grep "lines.*empty" demo_file

5. **Checking for full words, not for sub-strings using grep -w:**
   a) The following example is the regular grep where it is searching for “is”. When you search for “is”, without any option it will show out “is”, “his”, “this” and everything which has the substring “is”.
      - $ grep -i "is" demo_file
   b) The following example is the WORD grep where it is searching only for the word “is”. Please note that this output does not contain the line “This Line Has All Its First Character Of The Word With Upper Case”, even though “is” is there in the “This”, as the following is looking only for the word “is” and not for “this”.
      - $ grep -iw "is" demo_file
6. **Displaying lines before/after/around the match using grep -A, -B and -C:** Please create the following demo_text file for this example.
   - $ cat demo_text
   - 4. Vim Word Navigation
   -
   - You may want to do several navigation in relation to the words, such as:
   -
   - * e - go to the end of the current word.
   - * E - go to the end of the current WORD.
   - * b - go to the previous (before) word.
   - * B - go to the previous (before) WORD.
   - * w - go to the next word.
   - * W - go to the next WORD.
   -
   - WORD - WORD consists of a sequence of non-blank characters, separated with white space.
   - word - word consists of a sequence of letters, digits and underscores.
   -
   - Example to show the difference between WORD and word
   -
   - * 192.168.1.1 - single WORD

- * 192.168.1.1 - seven words.

a) -A is the option which prints the specified N lines after the match as shown below. The following example prints the matched line, along with the 3 lines after it.
   - $ grep -A 3 -i "example" demo_text

b) -B is the option which prints the specified N lines before the match. When you had option to show the N lines after match, you have the -B option for the opposite.
   - $ grep -B 2 "single WORD" demo_text

c) -C is the option which prints the specified N lines before the match. In some occasion you might want the match to be appeared with the lines from both the side. This options shows N lines in both the side(before & after) of match.
   - $ grep -C 2 "Example" demo_text

7. **Highlighting the search using GREP_OPTIONS:** As grep prints out lines from the file by the pattern / string you had given, if you wanted it to highlight which part matches the line, then you need to follow the following way. When you do the following export you will get the highlighting of the matched searches. In the following example, it will highlight all the this when you set the GREP_OPTIONS environment variable as shown below.
   - $ export GREP_OPTIONS='--color=auto' export GREP_COLOR= '0;31'
   - $ grep this demo_file
8. **Searching in all files recursively using grep -r:** When you want to search in all the files under the current directory and its sub directory. -r option is the one which you need to use. The following example will look for the string “ramesh” in all the files in the current directory and all it’s subdirectory.
   - $ grep -r "ramesh" *
9. **Invert match using grep -v:** You had different options to show the lines matched, to show the lines before match, and to show the lines after match, and to highlight match. So definitely You’d also want the option -v to do invert match. When you want to display the lines which does not matches the given string/pattern, use the option -v as shown below. This example will display all the lines that did not match the word “go”.
   - $ grep -v "go" demo_text

10. Display the lines which does not matches all the given pattern:
    - $ cat test-file.txt
    - a
    - b
    - c
    - d

    b) grep -v -e "a" -e "b" -e "c" test-file.txt
11. **Counting the number of matches using grep -c:**
    a) When you want to count that how many lines matches the given pattern/string, then use the option -c.
        - $ grep -c "go" demo_text

    b) When you want do find out how many lines matches the pattern
        - $ grep -c this demo_file

    c) When you want do find out how many lines that does not match the pattern
        - $ grep -v -c this demo_file
12. **Display only the file names which matches the given pattern using grep -l:** f you want the grep to show out only the file names which matched the given pattern, use the -l (lower-case L) option. When you give multiple files to the grep as input, it displays the names of file which contains the text that matches the pattern, will be very handy when you try to find some notes in your whole directory structure.
    a) $ grep -l this demo_*
13. **Show only the matched string:** By default grep will show the line which matches the given pattern/string, but if you want the grep to show out only the matched string of the pattern then use the -o option. It might not be that much useful when you give the string straight forward. But it becomes very useful when you give a regex pattern and trying to see what it matches as
        - $ grep -o "is.*line" demo_file
14. **Show the position of match in the line:** When you want grep to show the position where it matches the pattern in the file, use the following options as
        - $ cat temp-file.txt
        - 12345
        - 12345

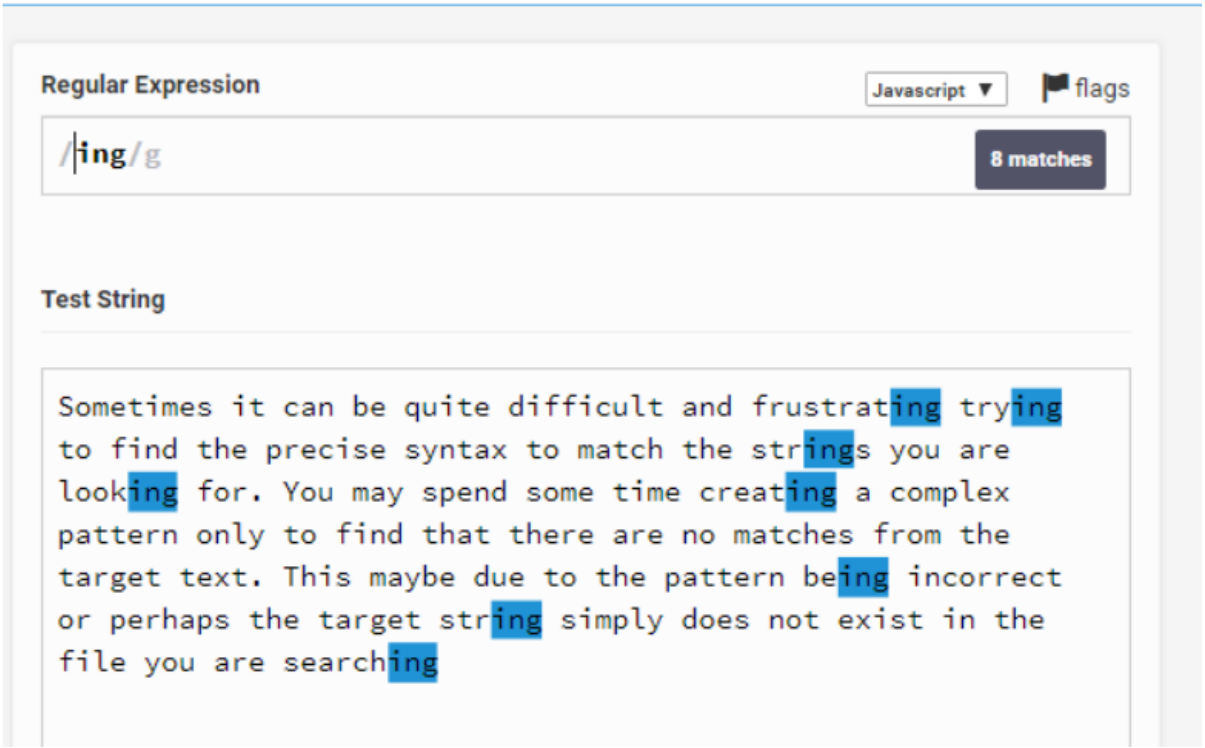
a) $ grep -o -b "3" temp-file.txt
    - Note: The output of the grep command above is not the position in the line, it is byte offset of the whole file.

15. **How line number while displaying the output using grep -n:** To show the line number of file with the line matched. It does 1-based line numbering for each file. Use -n option to utilize this feature.
    - $ grep -n "go" demo_text

Further Regular Expressions

Sometimes it can be quite difficult and frustrating trying to find the precise syntax to match the strings you are looking for. You may spend some time creating a complex pattern only to find that there are no matches from the target text. This may be due to the pattern being incorrect or perhaps the target string simply does not exist in the file you are searching.

However, there is an excellent on-line tool which demonstrates the operation of regular expressions. With **RegexPal** (http://regexpal.com/) users can enter some sample text then create their regular expression. Regexpal shows the matching strings highlighted in blue in real time.



Copy and paste the attached sample text below (an extract from a router configuration file) into the lower field of RegexPal and create your own regular expressions in the upper field to match the following requirements.

Download the file fs282.zip and then extract the content (directories and files) to your home directory.

1. Create a regular expression to match the word "password" (very simple). There should be two matches.
2. Create an expression to match the the string "int", but only if it occurs at the very start of a line. You will need to check the "multiline (m)" checkbox under the "Flags" menu. There should be five matches.
    a) Extend the expression so that the whole line is matched.
3. Create an expression to match digits only, (characters 0, 1, 2 etc.) There will be 318 matches.
4. Create an expression to match numbers only i.e. strings of digits.
    a) Modify this expression so that only numbers followed by a dot are matched. Remember "." Normally means "any character" in regex. Make sure that single dots on their own are not matched.
5. IP addresses are written as 4 numbers separated by dots. Create an expression to match all IP addresses or similar in the target text.
    a) Modify the expression so that the whole line containing an IP address is matched.

```
Regexpal.txt – sample text for
--------------------------
hostname Router1
!
enable secret 5 $1$mERr$hx5rVt7rPNoS4wqbXKX7m0
!
username ISP password 0 cisco123
!
banner motd $ Unauthorised access
Prohibited
 internal users only
 In case of emergency
 please contact Jim Jackson x2785.6.8.
j.jackson@napier.ac.uk 9 to 5 ONLY!
 @other times contact Barry Flint x6543.4.5
$
!
interface FastEthernet0/0.10
encapsulation dot1Q 10
```

```
ip address 192.168.10.254 255.255.255.0
ip nat inside
!
interface FastEthernet0/0.20
encapsulation dot1Q 20
ip address 168.20.254.17 255.255.255.0
ip nat inside
!
interface FastEthernet0/0.30
encapsulation dot1Q 30
ip address 192.168.30.254 255.255.255.0
ip access-group PC3 in
ip nat inside
!
int fa0/0.100
encapsulation dot1Q 100
ip address 192.168.100.254 255.255.255.0
ip nat inside
!
interface Serial0/0/0
ip address 172.16.1.2 255.255.255.252
encapsulation ppp
ppp authentication chap
ip nat outside
!
router rip
!
ip nat inside source list 10 interface Serial0/0/0 overload
ip classless
ip route 0.0.0.0 0.0.0.0 Serial0/0/0
!
ip access-list standard VTY-Local
permit host 192.168.10.50
deny any
access-list 10 permit 192.168.10.0 0.0.0.255
access-list 10 permit 192.168.20.0 0.0.0.255
access-list 10 permit 192.168.30.0 0.0.0.255
access-list 10 permit 192.168.100.0 0.0.0.255
ip access-list extended PC3
deny tcp host 192.168.30.50 host 81.95.37.133 eq ftp
deny ip host 192.168.30.50 192.168.10.0 0.0.0.255
permit ip any any
!
line con 0
line vty 0 4
access-class VTY-Local in
password cisco
login
!
!
```

end10:31 22/09/2015

Answers

1. password
2. ^int ^int.*
3. [0-9]
4. [0-9]* or [0-9]+ a. [0-9]+\.
5. [0-9]+\.[0-9]+\.[0-9]+\.[0-9]+
   a. .*([0-9]+\.){3}[0-9]+.*

Practical Linux grep command examples 2

Navigate to the directory fs282/path/web and use grep to search the word list for the following:

1. Find a word which contains the text “linu”.
2. Find the name of a place which contains the text “cs” and ends with an “n”.
   Note that the majority of dictionary words do not end at a new line but have a suffix in the form of "/M", "/S", "/MS" etc. So the character immediately following the word is a "/" slash.
3. What is the last word in the dictionary which contains the letters “c”, “s” and ”n” in order?
4. What is the first word in the dictionary which starts with “c” followed by “s” and then ”n”? (Hint: use “head” to see just the first lines of output)
5. Find a word which starts with an “s” and contains every vowel character “aeiou” in order.

Answers

1. botulinus 2. Tucson 3. yachtswomen 4. cabstand 5. sacrilegious

Practical Linux cmp command examples

Move into the fs282/mirror/drag/syntax/group/IMG_2012 directory and compare pairs of ".JPG" image files.

e.g cmp IMAG1403.JPG IMG_0079.JPG

Using cmp identify which of the following image files are identical.

IMAG1403.JPG	
IMG_0079.JPG	
IMG_1403.JPG	
IMG_2102.JPG	
IMG_FIVE.JPG	

Basic Forensics Example

A web services company has reported suspected illegal activity on it's company equipment. They run a small web server farm of 24 separate servers which should all be configured identically.

One of the servers is suspected of having been hacked and may have had it's configuration altered remotely by a third party.

The company has provided you with copies of the (httpd.conf) configuration files from each of the 24 servers for forensic examination.

Your task is to identify which server has been altered and what changes the hacker has implemented.

The files can be found in directory fs282/interface/www/html/servers

Scripting Challenges

Challenge 1

Create a script which implements a “safe” version of the Linux rm command which does not delete a file, but moves it into the ~/Trash directory.

The script should be called “safe_rm” and should accept the pathname to the file to be deleted as an argument. (In a similar way to rm).

i.e.

safe_rm sample.txt

Challenge 2

Create a restore script which will list any files in the ~/Trash directory and ask the user which one they wish to restore. This could simply mean asking the user if each one needs to be restored in turn.

When the user indicates a file should be restored they are prompted for the pathname of the directory it should be restored into.

The script should then move this file from the ~/Trash folder into the target directory.

The script should do the usual checking for exceptions.

- Check the path to the file to delete initially exists.
- Check the restore directory path is valid and exists.

Challenge 3

Modify the safe_rm script so that when a file is “deleted” (i.e. moved into~/Trash), it’s original location is recorded in some way.

Modify the restore script so that it will return the desired file by to its original location automatically, using the location saved when it was deleted.

Password Cracking Challenges

Challenge 1

The following md5sum values have been retrieved, it is known that the passwords which produced these are 4 character dictionary words.

Find the corresponding plaintext password from the md5sum hash list below.

Choose one at random.

This data is in the file “passwd.txt” available to download.

#Four Character Password Hashes

#==============================

6e8fe0f63ffbb20d6d202d5520f5051c

839d3df4c5a00150228773c33ee8a7d3

44e12bf0ebfaea3f1459d74467ed9188

23362fa661db4c6513d61e275598b1d2

4d746dee9dde402658f4b0bedeba53d6

3c848ffb6ac6347034feef85a11a9bb3

60191f83efc613ef4b1fbe36ecd62fca

576f3fbef2b9a15fa3b8a248a6948899

adcdbd510f53ad049fa1cdedd4499d4d

4e5330efbe8dad3f2aa1386cc4099555

e5509d515db158951ca12f8394421da9

39e62387013ad2ac1dc33bc844f7773f

8084133afa0cd22d688991ebadc51ab6

Challenge 2

The following list shows md5sum hashes from passwords which are all four letter words starting with a "g" and followed by two digits.

#e.g. "gone42"

Find the corresponding plaintext password

#Four Character NN Password Hashes

#Passwords are four letter words starting

#with a "g" and followed by two digits.

#e.g. "gate42"

#==============================

72c6fef159ba7c4dd7405cf6d57a43f8

65e96f238f2e56936e99f654863fbfac

0ee4dbb3ae421453fbfa56bf74107e1f

a539e180553e9b7563449f901eed487e

f27bf19d4aca248bbe32b1729e9cf744

6d14c444e4a57374886372a5afbe7b87

3465bd2c6d8328a25e6daf5055140c64

Challenge 3

Users john and dave use the following passwords, the md5sums are shown.

User ID	Plaintext Password	md5sum
john	jilt41	d7574ad1ef3780705105ffa48f80df3e
dave	leek54	7acb9627c1736539ec3d155c22f1af98

Create your own passwd.txt file with the following content.

john:7574ad1ef3780705105ffa48f80df3e

dave:7acb9627c1736539ec3d155c22f1af98

Create a script called passchk which prompts the user to enter their user ID (either john or dave) followed by their password.

The script then checks the supplied password's md5sum against that recorded for that user in the password.txt file and reports if there is agreement.

Challenge 4

Create a script called addpass which prompts for a user ID and a password and adds a new entry for that user in the passwd.txt file. The facility to add another user should be limited to an existing validated user.

Add a few new users and verify that each would be able to be validated using the passchk script.